Module 10 — Disjoint Sets / Union-Find

Sources: CLRS 4e ch. 19 (Data Structures for Disjoint Sets) · Skiena 3e §8.1.3 (The Union–Find Data Structure), §2.3.2 (inverse Ackermann), §15.5 (Set Data Structures catalog)

Big Idea

You have `n` elements partitioned into disjoint sets. You need two things: "**are `x` and `y` in the same set?**" and "**merge the sets containing `x` and `y`.**" That's it. That is the entire interface.

Skiena frames the tension exactly as he framed search trees: *the two obvious data structures each support only one of these operations efficiently.*

- **Label each element with its component number.** `same_component` is `O(1)` — compare two labels. But a merge must relabel every element of one side: `Θ(n)`.
- **Store the edges and re-derive components on demand.** Merging is `O(1)` — just record the edge. But every query needs a full graph traversal.

The union–find structure escapes by representing each set as a "**backwards**" **tree**: every node points at its *parent*, the root names the set. `FIND` walks up; `UNION` points one root at the other. Both are `O(height)`, and the entire subject is two one-line heuristics that crush the height:

1. **Union by rank (or size)** — always hang the *smaller/shorter* tree under the larger. Alone this gives height `≤ lg n`, hence `O(m lg n)`.
2. **Path compression** — after a `FIND`, point every node on the find path *directly at the root*. Alone this also helps; **together** they give `O(m α(n))`, where `α` is the inverse Ackermann function — **at most 4 for any `n` that can be written down in this universe.**

Two things make this chapter special.

First, the asymmetry between the code and the proof. As CLRS's own part introduction puts it: *"Representing each set as a simple rooted tree yields surprisingly fast operations… The amortized analysis that proves this time bound is as complex as the data structure is simple."* The implementation is about 15 lines. The proof is the hardest amortized analysis in the book.

Second, this bound is optimal. Tarjan proved a matching `Ω(m α(m,n))` lower bound for any structure meeting certain technical conditions, later strengthened by Fredman and Saks to a cell-probe lower bound. **You cannot do better.** That is rare and worth knowing.

Skiena's verdict, which is the right one: *"Union–find is a fast, simple data structure that every programmer should know."*

Remember months later: *parent pointers up, root names the set; link small under large; flatten on the way back.* `α(n) ≤ 4`*, so treat it as* `O(1)` *in practice and say "essentially linear" out loud.*

What You Should Be Able To Do After This Chapter

- Write union–find with both heuristics from memory in under two minutes.
- Explain why the naive linked-list union is `Θ(n²)` on a specific sequence, and how the weighted-union heuristic fixes it (Theorem 19.1).
- Give Skiena's doubling argument for why union by size alone bounds the height at `lg n`.
- State exactly what `rank` is (**an upper bound on height, not the height**) and why path compression doesn't update it.
- Define `A_k(j)` and `α(n)`, compute `A_0(1)…A_3(1)`, and say why `α(n) ≤ 4` for all practical `n`.
- Sketch the potential-method proof: `level(x)`, `iter(x)`, `φ_q(x)`, and why a `FIND` on a path of `s` nodes drops the potential by `≥ s − (α(n) + 2)`.
- Use union–find for connected components, Kruskal, cycle detection, offline LCA, offline minimum, and "equations/constraints consistency" problems.
- Explain what union–find **cannot** do (delete, split, un-union) and what you use instead.

Part 1 — The Interface and Its Application

The three operations

OPERATION	MEANING
`MAKE-SET(x)`	create a new set whose only member — and thus **representative** — is `x`
`UNION(x, y)`	unite the disjoint sets S_x and S_y; **destroys both** and creates their union. The representative may be any member; implementations typically pick one of the two old representatives
`FIND-SET(x)`	return a pointer to the representative of the unique set containing `x`

The representative is any designated member. The only contract is: *ask twice without modifying the set in between and you get the same answer.* (Some applications additionally demand a rule, e.g. "the smallest member" — that constrains which implementations you can use.)

The two parameters that appear in every bound:

- `n` = the number of `MAKE-SET` operations (the number of elements),
- `m` = the **total** number of operations, so $m \geq n$.

Two facts follow immediately and you should recite them: the first `n` operations are `MAKE-SET`s, so after them there are `n` singletons; and since each `UNION` reduces the number of sets by exactly 1, **at most `n - 1` `UNION` operations can ever occur.**

Application: connected components

```
CONNECTED-COMPONENTS(G)                SAME-COMPONENT(u, v)
1  for each vertex v ∈ G.V              1  if FIND-SET(u) == FIND-
SET(v)
2      MAKE-SET(v)                      2      return TRUE
3  for each edge (u,v) ∈ G.E            3  else return FALSE
4      if FIND-SET(u) ≠ FIND-SET(v)
5          UNION(u, v)
```

→ **C++ implementation:** A1 CONNECTED-COMPONENTS and SAME-COMPONENT

After processing all edges, two vertices are in the same connected component **iff** they're in the same set.

When to use this instead of DFS. CLRS is explicit: *when the edges are static, depth-first search computes the connected components faster*. DFS is $\Theta(V + E)$, plainly linear; union–find is $\Theta((V+E)\cdot\alpha(V))$. **Union–find wins when edges arrive dynamically** and components must be updated incrementally, because re-running DFS after every new edge is $\Theta(E(V+E))$.

Skiena's version of the same point: for Kruskal's algorithm you need `same_component(v1,v2)` and `merge_components(C1,C2)` interleaved, so a one-shot traversal is not an option.

> ***The general recognition rule:*** *union–find is the structure for* ***incremental (edge-adding) connectivity****. It has no answer for edge deletion, which is a genuinely harder problem (see the variants section).*

Part 2 — Two Representations

Linked-list representation (CLRS 19.2)

Each set is a linked list. The set object has `head` and `tail`; each element has a pointer to the next element **and a pointer back to the set object**. The representative is the first element.

- `MAKE-SET`: $O(1)$.
- `FIND-SET(x)`: follow `x`'s back-pointer to the set object, return the member at `head`. **$O(1)$.**
- `UNION(x, y)`: append `y`'s list to `x`'s (using `tail` to find the end) and **update the back-pointer of every element of `y`'s list**. $\Theta(|S_y|)$.

The $\Theta(n^2)$ sequence. Start with $x_1, \ldots, x_n$ and run n `MAKE-SET`s followed by $\text{UNION}(x_2,x_1), \text{UNION}(x_3,x_2), \ldots, \text{UNION}(x_n,x_{n-1})$ — always appending the big list onto the singleton. The i-th union updates i objects:

$$\Sigma_{i=1}^{n-1} i = \Theta(n^2)$$

with $m = 2n - 1$ operations, so **$\Theta(n)$ amortized per operation.** Catastrophic.

The weighted-union heuristic. Store each list's length; **always append the shorter list onto the longer.**

Theorem 19.1. With the linked-list representation and the weighted-union heuristic, a sequence of `m` operations, `n` of which are `MAKE-SET`, takes `O(m + n lg n)` time.

Proof skeleton — the argument you will reuse forever. Bound how many times a **single object** `x` has its back-pointer updated. Each time `x` is updated, `x` was in the **smaller** set, so the resulting set is at least **twice** as large as `x`'s old set. The first update lands `x` in a set of size `≥ 2`, the second `≥ 4`, the `k`-th `≥ 2^k`. Since the largest set has `≤ n` members, `x` is updated at most `⌈lg n⌉` times. Summing over all `n` objects: `O(n lg n)` for the unions. Each `MAKE-SET` and `FIND-SET` is `O(1)`, and there are `O(m)` of them. Total `O(m + n lg n)`. ∎

> ***This is the "small-to-large merging" argument**, and it is worth naming, because it appears far beyond union–find: merging two `std::set`s by iterating over the smaller one, merging DSU-on-tree ("small to large") in competitive programming, merging hash maps in a tree DP. Whenever you always iterate over the smaller side, each element is touched `O(lg n)` times in total.*

Disjoint-set forests (CLRS 19.3, Skiena 8.1.3)

Represent each set as a **rooted tree where each node points only to its parent.** The root is its own parent and is the representative.

- `MAKE-SET(x)`: a one-node tree.
- `FIND-SET(x)`: follow parent pointers to the root. The nodes visited form the **find path**.
- `UNION(x, y)`: make one root point to the other.

Without heuristics this is no better than the list version — `n−1` unions can build a single chain of `n` nodes and every `FIND` costs `Θ(n)`.

Heuristic 1 — union by rank / union by size

Union by size (Skiena's version). Keep `size[v]` = number of elements in `v`'s subtree; on a union, **make the smaller tree the subtree of the bigger one**.

Skiena's justification is the cleanest sentence on the subject: "The heights of all the nodes in the root subtree stay the same, but the height of the nodes merged into this tree all increase by one. Thus, merging in the smaller tree leaves the height unchanged on the larger set of vertices."

And his height analysis is the doubling argument again: what is the **smallest** tree of height `h`? A single node has height 1. The smallest height-2 tree has 2 nodes (union of two singletons) — merging in more singletons doesn't increase the height, they just become children. Only merging two height-2 trees gives height 3, with `≥ 4` nodes. **You must double the node count to gain a unit of height**, and you can only double `lg n` times. So **height `≤ lg n`**, and both unions and finds are `O(lg n)`.

Union by rank (CLRS's version). Instead of the size, keep `x.rank`, **an upper bound on the height of `x`**. `MAKE-SET` sets `rank = 0`. On a `LINK`:

- unequal ranks → the **higher-rank** root becomes the parent; **no rank changes**;
- equal ranks → pick either as parent and **increment its rank by 1**.

Why rank rather than size? CLRS: *"Rather than explicitly keeping track of the size of the subtree rooted at each node, we'll adopt an approach that eases the analysis."* The two perform essentially identically in practice.

> ***The subtlety that trips people up:*** *once path compression is in play,* ***`rank` is no longer the height*** *— compression shortens trees without touching ranks.* `rank` *remains a valid upper bound on height, which is all the proof needs.* ***Path compression does not change any ranks.***

Heuristic 2 — path compression

During `FIND-SET`, make **every node on the find path point directly at the root.**

```
FIND-SET(x)
1  if x ≠ x.p                       // not the root?
2      x.p = FIND-SET(x.p)          // the root becomes the parent
3  return x.p                       // return the root
```

→ **C++ implementation:** A2 FIND-SET (with path compression)

Three lines. It is a **two-pass method**: the recursion walks *up* the find path to the root, and the unwinding walks back *down* relinking each node. Note the elegance: `FIND-SET` returns `x.p` in line 3 in both cases — if `x` is the root, line 2 is skipped and `x.p` is `x` itself.

Skiena's summary: *"Shrinking the path traced after each find, by explicitly pointing each path node directly to the root, is called path compression and reduces the tree to almost constant height."*

Full pseudocode

```
MAKE-SET(x)                UNION(x, y)                        LINK(x,
y)
1  x.p = x                 1  LINK(FIND-SET(x),               1  if
x.rank > y.rank
2  x.rank = 0                        FIND-SET(y))             2
y.p = x
                                                              3  else
x.p = y
                                                              4      if
x.rank == y.rank
                                                              5
y.rank = y.rank + 1
```

→ **C++ implementation:** A3 MAKE-SET, UNION and LINK

Effect of the heuristics

HEURISTICS	BOUND FOR `m` OPERATIONS, `n` OF THEM `MAKE-SET`
neither	`Θ(m n)` worst case (a chain)
union by rank only	`O(m lg n)`, and this is **tight** (Ex. 19.3-3)
path compression only	`Θ(n + f·(1 + log_{2+f/n} n))`, where `f` is the number of `FIND-SET`s
both	**`O(m α(n))`** — and `α(n) ≤ 4` always in practice

Verified: on `n = 65 536` elements after 262 144 random unions —

- union by rank **without** compression: measured tree height **7** (bound `lg n = 16`), and max rank **7 ⩽ ⌊lg n⌋`.
- union by rank **with** compression: **average find-path length 1.0006, maximum 2.** The forest is essentially flat.

Part 3 — Why `α(n) ≤ 4` (CLRS 19.4)

The very quickly growing function

For integers $j, k \geq 0$:

$$A_k(j) = \begin{cases} j + 1 & \text{if } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{if } k \geq 1 \end{cases} \tag{19.1}$$

where $A^{(i)}$ is **functional iteration**: $A^{(0)}(j) = j$ and $A^{(i)}(j) = A(A^{(i-1)}(j))$. Call k the **level**. $A_k(j)$ strictly increases in both arguments.

Lemma 19.2. For any integer $j \geq 1$: $A_1(j) = 2j + 1$.
Proof. Induction on i gives $A_0^{(i)}(j) = j + i$ (base $A_0^{(0)}(j) = j$; step $A_0(A_0^{(i-1)}(j)) = (j + i - 1) + 1$). Then $A_1(j) = A_0^{(j+1)}(j) = j + (j+1) = 2j + 1$. ∎

Lemma 19.3. For any integer $j \geq 1$: $A_2(j) = 2^{j+1}(j+1) - 1$.
Proof. Induction on i gives $A_1^{(i)}(j) = 2^i(j+1) - 1$. Then $A_2(j) = A_1^{(j+1)}(j) = 2^{j+1}(j+1) - 1$. ∎

Now watch it explode at $j = 1$:

k	$A_k(1)$
0	$1 + 1 = 2$
1	$2 \cdot 1 + 1 = 3$
2	$2^2 \cdot 2 - 1 = 7$
3	$A_2^{(2)}(1) = A_2(A_2(1)) = A_2(7) = 2^8 \cdot 8 - 1 = 2^{11} - 1 = 2047$
4	$A_3(A_3(1)) = A_3(2047) = A_2^{(2048)}(2047) \geq A_2(2047) = 2^{2^{048}} \cdot 2048 - 1 = 2^{2^{059}} - 1 > 2^{2^{056}} = 16^{5^{14}} \gg 10^{80}$

10^{80} is the estimated number of atoms in the observable universe.

Verified: $A_k(1) = 2, 3, 7, 2047$ for $k = 0..3$ computed directly from (19.1), and both closed forms checked against direct iteration for $j = 1..12$.

The very slowly growing inverse

```
α(n) = min { k : A_k(1) ≥ n }
   (19.2)
```

— *the lowest level* `k` *at which* `A_k(1)` *reaches* `n`. Reading the table backwards:

```
          ⎧ 0   for 0 ≤ n ≤ 2
          ⎪ 1   for n = 3
α(n)  =   ⎨ 2   for 4 ≤ n ≤ 7
          ⎪ 3   for 8 ≤ n ≤ 2047
          ⎩ 4   for 2048 ≤ n ≤ A₄(1)
```

`α(n) > 4` only for `n` so large that "astronomical" understates it. Skiena's gloss: *"It is sufficient to think of this as geek talk for the slowest growing complexity function... The value of `α(n)` is smaller than 5 for any value of `n` that can be written in this physical universe."*

Say it correctly: `O(m α(n))` is **not** `O(m)`. It is genuinely superlinear as a function. It is just that the superlinearity never manifests. In an interview: *"`O(α(n))` amortized per operation, which is at most 4 for any input that fits in a computer, so effectively constant — but it isn't literally constant."*

Properties of ranks

Lemma 19.4. For all nodes `x`: `x.rank ≤ x.p.rank`, with **strict** inequality when `x` is not a root. `x.rank` starts at 0, increases while `x` is a root, and **once `x` stops being a root it never changes again**. `x.p.rank` monotonically increases over time.

Corollary 19.5. Going up any simple path toward a root, **ranks strictly increase.**

Lemma 19.6. Every node has rank at most `n − 1`.
Proof. Ranks start at 0 and increase only on `LINK`; there are `≤ n−1` `LINK`s and each raises at most one rank by 1. ∎

Exercise 19.4-2 (sharper, and the one to remember): every node has rank at most `⌊lg n⌋`. *(A node of rank `r` roots a subtree of `≥ 2^r` nodes — the same doubling argument as Skiena's.)* CLRS only needs the loose Lemma 19.6 for the proof.

Verified: max rank after 262 144 unions on 65 536 elements was **7**, well under `⌊lg n⌋ = 16`.

The potential-method proof (structure, not every line)

The technique is M09's potential method. Two preliminaries:

Lemma 19.7 (reduce `UNION` to `LINK`). Convert each `UNION` into two `FIND-SET`s plus one `LINK`. Then `m′ ≤ m ≤ 3m′`, so `m = Θ(m′)` and an `O(m α(n))` bound on the converted sequence gives `O(m′ α(n))` on the original. *(This is a routine but necessary step — it lets the analysis treat `LINK` as taking two roots.)*

The potential. `Φ_q = Σ_x φ_q(x)`, with `Φ₀ = 0` and `Φ_q ≥ 0` always. For a node `x` after `q` operations:

- If `x` is a **root**, or `x.rank = 0`: `φ_q(x) = α(n) · x.rank`.
- Otherwise (`x` a non-root with `x.rank ≥ 1`), define two auxiliary functions:

```
level(x) = max { k : x.p.rank ≥ A_k(x.rank) }
   (19.3)
iter(x)  = max { i : x.p.rank ≥ A_{level(x)}^{(i)}(x.rank) }
    (19.5)

φ_q(x) = (α(n) − level(x)) · x.rank − iter(x)
   (19.7)
```

Read these two functions as a two-digit odometer measuring how far `x` has been flattened. `level(x)` asks *how many levels of the `A` hierarchy separate `x`'s rank from its parent's rank*; `iter(x)` refines that with *how many iterations of `A_{level(x)}` fit in the gap*. Both are bounded:

```
0 ≤ level(x) < α(n)              (19.4)
1 ≤ iter(x) ≤ x.rank             (19.6)
```

and crucially **`level(x)` monotonically increases over time** (a non-root's own rank is frozen by Lemma 19.4 while its parent's rank only grows, so the gap widens), and while `level(x)` is unchanged, `iter(x)` can only increase.

Lemma 19.8 / Corollary 19.9. `0 ≤ φ_q(x) ≤ α(n)·x.rank`, with strict inequality for a non-root of positive rank. *(Maximize `level` and `iter` for the lower bound; minimize them for the upper.)*

Lemma 19.10 — the engine of the whole proof. For a non-root `x` and a `LINK` or `FIND-SET`: `φ_q(x) ≤ φ_{q−1}(x)`; and if `x.rank ≥ 1` and **either `level(x)` or `iter(x)` changes**, then `φ_q(x) ≤ φ_{q−1}(x) − 1`.
Why: `x.rank` and `α(n)` are frozen. If `level` is unchanged, `iter` can only rise, dropping `φ` by `≥ 1`. If `level` rises by `≥ 1`, the term `(α(n) − level(x))·x.rank` drops by `≥ x.rank`, while `iter(x)` can rise back by at most `x.rank − 1` — a **net drop of at least 1**.

The three amortized costs:

- **Lemma 19.11 — `MAKE-SET` is `O(1)` amortized.** It creates a rank-0 node with `φ = 0`; nothing else changes, so `Φ_q = Φ_{q−1}`.
- **Lemma 19.12 — `LINK` is `O(α(n))` amortized.** Actual cost `O(1)`. Only `x`, `y`, and `y`'s prior children can change potential. `y`'s old children can't increase (Lemma 19.10). `x` was a root and becomes a non-root, so its potential **decreases** (Corollary 19.9). `y` stays a root, and its rank either stays or rises by 1, so `φ(y)` rises by at most `α(n)`. Total increase `≤ α(n)`.
- **Lemma 19.13 — `FIND-SET` is `O(α(n))` amortized.** Actual cost `O(s)` on a find path of `s` nodes. No node's potential increases. And **at least `max{0, s − (α(n) + 2)}` nodes on the path have their potential drop by `≥ 1`.**

  Why that count: the nodes that might **not** drop are only: the first node (if rank 0), the last node (the root), and, **for each `k = 0, 1, …, α(n) − 1`, the last node `w` on the path with `level(w) = k`.** That's `2 + α(n)` exceptions. Every *other* node `x` on the path has positive rank and is followed on the path by some non-root `y` with `level(y) = level(x)`; chaining the definitions of `level` and `iter` with Corollary 19.5 shows `y.p.rank ≥ A_k^{(i+1)}(x.rank)`, so after compression (which makes `x.p = y.p`'s root) `iter(x)` rises from `i` to at least `i+1`, and Lemma 19.10 fires.

  Amortized cost `= O(s) − (s − (α(n)+2)) = O(α(n))`, after scaling the potential units to swallow the constant hidden in `O(s)`.

Theorem 19.14. A sequence of `m` `MAKE-SET`, `UNION`, `FIND-SET` operations, `n` of which are `MAKE-SET`, runs on a disjoint-set forest with union by rank and path compression in **`O(m α(n))`** time. ∎

What to actually retain from §19.4 for an interview: the bound, `α(n) ≤ 4`, that the proof is a potential argument where *each node's potential drops by at least 1 every time path compression genuinely shortens its distance-in-`A`-levels to the root*, and that the exceptions on a find path number only `α(n) + 2`. Nobody will ask you to reproduce Lemma 19.13.

The lower bound

Tarjan proved $\Omega(m\ \alpha(m,n))$ is **required** for any disjoint-set structure satisfying certain technical conditions. Fredman and Saks generalized it to the cell-probe model: $\Omega(m\ \alpha(m,n))$ words of $\theta(\lg n)$ bits must be accessed in the worst case. **Union–find with both heuristics is optimal**, not merely good.

*(Also from the chapter notes: Goel et al. proved that linking disjoint-set trees **randomly** — no rank, no size, just a coin flip — achieves the same asymptotic bound. And Gabow–Tarjan show that in certain restricted applications you can get a true $O(m)$.)*

Part 4 — C++ Implementations

The one you should be able to type from memory

```
#include <algorithm>
#include <numeric>
#include <vector>

class DisjointSet {
public:
    explicit DisjointSet(int n) : parent_(n), rank_(n, 0),
componentCount_(n) {
        iota(parent_.begin(), parent_.end(), 0);          // MAKE-
SET for every element
    }

    // Two-pass FIND-SET: walk to the root, then point every node
on the
    // find path directly at it.  Iterative, so no O(depth) stack.
    int find(int x) {
        int root = x;
        while (parent_[root] != root) root = parent_[root];
        while (parent_[x] != root) { const int nextOnPath =
parent_[x]; parent_[x] = root; x = nextOnPath; }
        return root;
    }

    // LINK the two roots: smaller rank hangs off larger; on a tie,
bump.
```

```
    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;
        // After this swap, rootX is the SHORTER tree and hangs
under rootY.
        if (rank_[rootX] > rank_[rootY]) swap(rootX, rootY);
        parent_[rootX] = rootY;
        if (rank_[rootX] == rank_[rootY]) ++rank_[rootY];
        --componentCount_;
        return true;
    }

    bool sameSet(int x, int y) { return find(x) == find(y); }
    int  count() const { return componentCount_; }
    int  rankOf(int x) const { return rank_[x]; }

private:
    vector<int> parent_, rank_;
    int componentCount_;
};
```

Implementation notes.

- `find` is written **iteratively** on purpose. CLRS's recursive version is beautiful but a deep tree before the first compression can blow the stack; the two-pass loop has the same effect with `O(1)` stack.
- `unite` returns `bool` — `true` if a merge happened. That return value is what Kruskal's algorithm and cycle detection consume.
- `sets_` maintains the component count for free.
- Elements are `0..n-1`. If your problem has string or coordinate keys, map them to indices first — do **not** build a pointer-based version.

Union by size with path halving (Skiena's shape, one-pass find)

```
#include <algorithm>
#include <numeric>
#include <vector>

class DisjointSetBySize {
public:
```

```
    explicit DisjointSetBySize(int n) : parent_(n), size_(n, 1),
componentCount_(n) {
        iota(parent_.begin(), parent_.end(), 0);
    }

    // One-pass find: halve the path as we climb (every other node
is relinked).
    int find(int x) {
        while (parent_[x] != x) { parent_[x] = parent_[parent_[x]];
x = parent_[x]; }
        return x;
    }

    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;
        // After this swap, rootX is the SMALLER tree and becomes
the child.
        if (size_[rootX] > size_[rootY]) swap(rootX, rootY);
        parent_[rootX] = rootY;
        size_[rootY] += size_[rootX];
        --componentCount_;
        return true;
    }

    bool sameSet(int x, int y) { return find(x) == find(y); }
    int  sizeOf(int x) { return size_[find(x)]; }
    int  count() const { return componentCount_; }

private:
    vector<int> parent_, size_;
    int componentCount_;
};
```

Rank vs. size, halving vs. full compression — the honest comparison:

	UNION BY RANK	UNION BY SIZE
extra field	`int rank` (`≤ ⌊lg n⌋`, so 5–6 bits suffice — Ex. 19.4-3)	`int size` (needs `lg n` bits)
gives you	nothing extra	**`sizeOf(x)` for free**, which many problems want
asymptotics	identical `O(m α(n))`	identical

	FULL PATH COMPRESSION (2-PASS)	PATH HALVING / SPLITTING (1-PASS)
passes over the find path	2	1
flatness achieved	perfect (all nodes → root)	near-perfect
asymptotics	`O(m α(n))`	`O(m α(n))`
in practice	slightly more pointer writes	often marginally faster; the chapter notes cite Tarjan–van Leeuwen on these "one-pass methods" offering better constant factors

In competitive programming, use union by size with path halving — shorter to type and gives you component sizes. **In an interview, either is fine; say which heuristics you're using and why.**

Verified: both implementations agreed with a brute-force relabeling array over 20 000 randomized operations on 3000 elements, ending at 35 components.

Weighted DSU — relations, not just membership (CLRS Problem 19-2 generalized)

CLRS Problem 19-2 ("depth determination") introduces the key idea: store in each node a **pseudodistance** `v.d` such that *the sum of pseudodistances along the path from `v` to its set's root equals the quantity you actually want to know about `v`*. Path compression must then **accumulate** these offsets as it relinks.

Generalized, this is the **weighted / potential DSU**: maintain `value[x] − value[root(x)]` for every `x`, so you can answer "what is `value[y] − value[x]`?" for any two elements in the same set, and detect contradictions.

```
#include <algorithm>
```

```cpp
#include <numeric>
#include <utility>
#include <vector>

class WeightedDSU {
public:
    explicit WeightedDSU(int n) : parent_(n), rank_(n, 0),
offsetToParent_(n, 0) {
        iota(parent_.begin(), parent_.end(), 0);
    }

    // Returns {root, value[x] - value[root]}.
    pair<int, long long> find(int x) {
        if (parent_[x] == x) return {x, 0};
        const auto parentInfo = find(parent_[x]);          //
recursive: compresses on unwind
        parent_[x] = parentInfo.first;
        offsetToParent_[x] += parentInfo.second;
        return {parent_[x], offsetToParent_[x]};
    }

    // Assert value[y] - value[x] == d. Returns false on
contradiction.
    bool relate(int x, int y, long long wantedGap) {
        auto infoX = find(x), infoY = find(y);
        if (infoX.first == infoY.first) return infoY.second -
infoX.second == wantedGap;
        int rootX = infoX.first, rootY = infoY.first;
        // offsetX = value[x] - value[rootX];  offsetY = value[y] -
value[rootY]
        long long offsetX = infoX.second, offsetY = infoY.second;
        // want value[y] - value[x] = wantedGap
        //   =>  value[rootY] - value[rootX] = offsetX + wantedGap
- offsetY
        long long rootGap = offsetX + wantedGap - offsetY;
        // After this swap, rootX is the shorter tree and hangs
under rootY.
        if (rank_[rootX] > rank_[rootY]) { swap(rootX, rootY);
swap(offsetX, offsetY); rootGap = -rootGap; }
        parent_[rootX] = rootY;
        offsetToParent_[rootX] = -rootGap;                  //
value[rootX] - value[rootY]
```

```
        if (rank_[rootX] == rank_[rootY]) ++rank_[rootY];
        return true;
    }

    // value[y] - value[x] if x and y are in the same set
    bool diff(int x, int y, long long& gap) {
        auto infoX = find(x), infoY = find(y);
        if (infoX.first != infoY.first) return false;
        gap = infoY.second - infoX.second;
        return true;
    }

private:
    vector<int> parent_, rank_;
    vector<long long> offsetToParent_;
};
```

What this solves: *"Given constraints `x_j - x_i = d`, are they consistent?"*; **incremental bipartiteness** (use offsets mod 2 — a graph is bipartite iff no edge relates two nodes with the same parity); "food chain"-style relation puzzles (offsets mod 3); and CLRS Problem 19-2's depth determination (offset = depth contribution, `GRAFT` = a `LINK` that sets the right pseudodistance).

Note this `find` is **recursive on purpose** — the offset accumulation is naturally expressed on the unwind, exactly like CLRS's `FIND-SET`. With both heuristics the depth before compression is `O(lg n)`, so the stack is safe.

Verified: over 5000 randomized operations on 500 elements with a hidden ground-truth assignment, every consistent fact was accepted and every reported difference matched the truth; a hand-built contradiction (`0→1 = 5`, `1→2 = 3`, then claiming `0→2 = 9`) was rejected.

Rollback DSU — undo, at the cost of compression

Path compression is **destructive**: it rewrites parent pointers of nodes you weren't asked about, so you cannot undo a union by remembering one pointer. If you need `undo`, **drop path compression** and rely on union by size alone (`O(lg n)` per find, still fine).

```
#include <algorithm>
#include <cassert>
#include <numeric>
```

```cpp
#include <vector>

class RollbackDSU {
public:
    explicit RollbackDSU(int n) : parent_(n), size_(n, 1),
componentCount_(n) {
        iota(parent_.begin(), parent_.end(), 0);
    }

    int find(int x) const { while (parent_[x] != x) x = parent_[x];
return x; }   // O(lg n), no compression

    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) { history_.push_back({-1, -1, 0});
return false; }
        if (size_[rootX] > size_[rootY]) swap(rootX, rootY);
        history_.push_back({rootX, rootY, size_[rootY]});
        parent_[rootX] = rootY;
        size_[rootY] += size_[rootX];
        --componentCount_;
        return true;
    }

    void rollback() {
        assert(!history_.empty());
        const UnionRecord record = history_.back();
        history_.pop_back();
        if (record.child < 0) return;                        // the
union was a no-op
        parent_[record.child] = record.child;
        size_[record.parent] = record.parentSize;
        ++componentCount_;
    }

    int count() const { return componentCount_; }

private:
    struct UnionRecord { int child, parent, parentSize; };
    vector<int> parent_, size_;
    vector<UnionRecord> history_;
    int componentCount_;
```

```
};
```

Note that a no-op union still pushes a record, so `rollback()` pairs one-to-one with `unite()` — otherwise the caller has to track which unions "took", which is a reliable source of bugs.

Verified: interleaved unions and rollbacks over 3000 steps, cross-checked every 100 steps against a `DisjointSet` rebuilt from scratch on the surviving edge list — component counts and 50 random connectivity queries matched every time.

> *Outside / Engineering Context — offline dynamic connectivity*
>
> *Rollback DSU is the workhorse of **offline dynamic connectivity**: edges are inserted and deleted over time. Build a segment tree over the time axis, put each edge into the `O(lg T)` nodes covering its alive-interval, then DFS the segment tree adding edges on the way down and **rolling back** on the way up. Total `O((V + E) lg V lg T)`. There is no known simple online structure for edge deletion — that is why deletion is the boundary of what union–find can do.*

Part 5 — Applications

APPLICATION	HOW UNION–FIND IS USED
Kruskal's MST (M14)	sort edges; add `(u,v)` iff `unite(u,v)` returns `true`. `O(m lg m)` dominated by the sort
Connected components / cycle detection	`unite` returns `false` ⟺ the edge closes a cycle
Percolation / flood fill on grids	index cells as `r·C + c`; union neighbors; add virtual "top" and "bottom" nodes
Equivalence of names/accounts/strings	map to indices, union on each "these are the same" fact
Incremental bipartiteness / 2-coloring	weighted DSU with offsets mod 2
Difference constraints `x_j − x_i = d`	weighted DSU with integer offsets
Offline LCA (Tarjan)	CLRS Problem 19-3, below
Offline minimum	CLRS Problem 19-1, below
"Merge intervals" / "next free slot"	`find(i)` = smallest index `≥ i` not yet used; union `i` to `i+1` when consumed
Image segmentation, maze generation, clustering	classic uses; single-linkage clustering *is* Kruskal
Compiler / type inference unification	the "union" in Hindley–Milner unification is literally this structure

Tarjan's offline LCA (CLRS Problem 19-3)

Given a rooted tree and a set `P` of query pairs, find every pair's lowest common ancestor — **in one DFS**, `O((n + q) α(n))`.

```
LCA(u)
 1  MAKE-SET(u)
 2  FIND-SET(u).ancestor = u
 3  for each child v of u in T
 4      LCA(v)
 5      UNION(u, v)
 6      FIND-SET(u).ancestor = u
 7  u.color = BLACK
 8  for each node v such that {u, v} ∈ P
 9      if v.color == BLACK
10          print LCA of u and v is FIND-SET(v).ancestor
```

→ **C++ implementation:** A5 Tarjan's offline LCA

Why it works. The invariant is: *at the moment* `LCA(u)` *is called, the number of sets equals* `u`*'s depth* — the sets are exactly the "already-finished subtrees hanging off the root path to `u`", each labeled with the ancestor on that root path. So when we finish `u` and find a black partner `v`, `FIND-SET(v).ancestor` is precisely the deepest ancestor of `u` that is also an ancestor of `v`. Line 10 fires exactly once per pair (the second of the two to turn black triggers it).

```
#include <functional>
#include <utility>
#include <vector>

static vector<int> offlineLCA(const vector<vector<int>>& children,
                              int root,
                              const vector<pair<int, int>>&
queries) {
    const int n = (int)children.size();
    vector<vector<pair<int, int>>> byNode(n);   // (other, query
index)
    for (int i = 0; i < (int)queries.size(); ++i) {
        byNode[queries[i].first].push_back({queries[i].second, i});
        byNode[queries[i].second].push_back({queries[i].first, i});
    }
    DisjointSet dsu(n);
    vector<int> ancestor(n), answer(queries.size(), -1);
    vector<char> black(n, 0);

    function<void(int)> dfs = [&](int u) {
        ancestor[dsu.find(u)] = u;
        for (int v : children[u]) {
            dfs(v);
            dsu.unite(u, v);
            ancestor[dsu.find(u)] = u;      // the merged set is
still "under u"
        }
        black[u] = 1;
        for (const auto& queryPair : byNode[u])
            if (black[queryPair.first]) answer[queryPair.second] =
ancestor[dsu.find(queryPair.first)];
    };
    dfs(root);
```

```
    return answer;
}
```

Verified: on a random 2000-node tree with 5000 query pairs, every answer matched a brute-force depth-climbing LCA.

The catch, and why you should know it: this is **offline** — all queries must be known up front. For online LCA you use binary lifting or Euler tour + sparse table (M13/M18 territory).

Offline minimum (CLRS Problem 19-1)

Given a sequence of `INSERT`s (each key in `1..n` inserted exactly once) interleaved with `m` `EXTRACT-MIN`s, determine what each `EXTRACT-MIN` returns — allowed to see the whole sequence first.

The trick: break the sequence into $I_1, E, I_2, E, \ldots, I_m, E, I_{m+1}$ and let K_j be the keys inserted in block j. Then process keys **in increasing order** — key i is returned by the *first surviving extraction at or after* the block it was inserted into. Once an extraction is used, merge its block into the next surviving one. **Union–find gives "the next surviving block" in α-time.**

```
OFFLINE-MINIMUM(m, n)
1  for i = 1 to n                          // keys in increasing
order
2      determine j such that i ∈ K_j
3      if j ≠ m + 1
4          extracted[j] = i
5          let l be the smallest value greater than j for which K_l
exists
6          K_l = K_j ∪ K_l, destroying K_j
7  return extracted
```

→ **C++ implementation:** A6 OFFLINE-MINIMUM

This is the "**next free slot**" idiom in disguise, and that idiom is worth memorizing on its own: *maintain `find(i)` = the smallest index `≥ i` that is still available; when you consume `i`, `unite(i, i+1)`.*

Verified: reproduces CLRS's own example (`4, 8, E, 3, E, 9, 2, 6, E, E, E, 1, 7, E, 5` → `extracted = ⟨4, 3, 2, 6, 8, 1⟩`) and matches a simulated priority queue on 200 randomized sequences.

Augmentation: `PRINT-SET` (Exercise 19.3-4)

Add **one** attribute per node — a `next` pointer forming a **circular linked list** of each set's members. `MAKE-SET` makes a self-loop; `UNION` splices the two circles by swapping the two roots' `next` pointers (an `O(1)` operation). `PRINT-SET(x)` walks the circle from `x` back to `x`, linear in the set size. **All other operations are unaffected.**

This is the standard way to answer "list everything in this component" without a `Θ(n)` scan, and it's a nice small example of the augmentation discipline from M08.

Common Mistakes

MISTAKE	CONSEQUENCE
`p_[x] = y` instead of `p_[find(x)] = find(y)` in `unite`	Silently wrong: you attach a non-root, orphaning a subtree
Comparing `p_[x] == p_[y]` instead of `find(x) == find(y)`	Wrong answers whenever the trees have depth > 1
Union by rank **and** using `rank` as the height after compression	`rank` is only an *upper bound* on height once compression runs
Updating ranks during `FIND-SET`	Breaks the analysis; CLRS: *"Path compression does not change any ranks"*
Recursive `find` on adversarial input before any compression	Stack overflow at `n ≈ 10^5`+ if you skipped union by rank/size
Adding rollback **and** keeping path compression	Compression rewrites unrecorded pointers; undo is impossible. Drop compression
Expecting to delete an element or split a set	Union–find has no such operation. See offline dynamic connectivity
Forgetting `unite` may be a no-op, then decrementing the component count anyway	Component count drifts
Using union–find for **static** connected components	DFS/BFS is `Θ(V+E)` and simpler; save DSU for incremental edges or Kruskal
Saying "`O(1)` amortized"	It's `O(α(n))`. Say "effectively constant, at most 4 in practice"

Recognition Patterns

Reach for union–find when the problem says any of these:

- **"are these two connected / in the same group / equivalent?"** with **edges or equalities arriving over time**
- **"how many groups are there?"** / **"size of the group containing x"**
- **"detect a cycle in an undirected graph as edges are added"**
- **"merge accounts / emails / names that refer to the same entity"**
- **"is this set of equations `x - y = d` consistent?"** → weighted DSU
- **"minimum spanning tree"** → Kruskal → DSU

- **"find the next available slot / the next unfilled position"** → the `unite(i, i+1)` idiom
- **grid problems: islands, percolation, flood-filling with incremental additions** (note: for a *single static* flood fill, plain BFS/DFS is simpler)

The negative signals — union–find is the *wrong* answer when:

- edges are **deleted** (needs offline segment-tree-over-time, or a much heavier online structure);
- the graph is **directed** and you need strong connectivity (that's Tarjan/Kosaraju SCC, M13);
- you need **paths**, not just connectivity (DSU knows *whether* two nodes are connected, never *how*);
- the connectivity question is asked once on a static graph (DFS).

Complexity Summary

IMPLEMENTATION	MAKE-SET	FIND-SET	UNION	m OPERATIONS TOTAL
Linked list, naive union	$O(1)$	$O(1)$	$O(n)$	$\Theta(n^2)$ worst case
Linked list + weighted union	$O(1)$	$O(1)$	$O(n)$ worst / $O(\lg n)$ amortized	$O(m + n \lg n)$
Forest, no heuristics	$O(1)$	$O(n)$	$O(n)$	$\Theta(mn)$
Forest + union by rank	$O(1)$	$O(\lg n)$	$O(\lg n)$	$\Theta(m \lg n)$ (tight)
Forest + path compression only	$O(1)$	—	—	$\Theta(n + f(1 + \log_{2+f/n} n))$
Forest + both	$O(1)$	$O(\alpha(n))$ am.	$O(\alpha(n))$ am.	**$O(m\,\alpha(n))$** — optimal

Space: $\Theta(n)$ — two `int` arrays. `rank` needs only $\lceil \lg \lg n \rceil$-ish bits (Ex. 19.4-3: since $\text{rank} \le \lfloor \lg n \rfloor$, 6 bits cover n up to 2^{63}), so a packed implementation stores parent and rank in one word.

One-Page Recall

- **Interface:** `MAKE-SET`, `UNION`, `FIND-SET`. Parameters: `n` = elements, `m` = total operations, `m ≥ n`. At most `n − 1` unions ever.
- **Representation:** a forest of "backwards" trees — each node points at its parent, the root names the set.
- **Union by rank/size:** hang the shorter/smaller under the taller/bigger. **Alone: height `≤ lg n`, `O(m lg n)`, tight.** Skiena's proof: you must *double the node count* to add a unit of height, and you can only double `lg n` times.
- **Path compression:** in `FIND-SET`, relink every node on the find path directly to the root. Three lines, two passes, **changes no ranks**.
- **Together: `O(m α(n))`**, and this is **optimal** (Tarjan; Fredman–Saks).
- **`α(n)`:** `A_k(j) = j+1` for `k=0`, else `A_{k−1}^{(j+1)}(j)`; $A_1(j) = 2j+1$, $A_2(j) = 2^{j+1}(j+1) - 1$. $A_k(1) = 2, 3, 7, 2047, > 10^{80}$ for `k = 0,1,2,3,4`. `α(n) = min{k : A_k(1) ≥ n} ≤ 4` for every `n` you will ever see. **`O(m α(n))` is superlinear in theory, linear in practice.**
- **Rank facts:** ranks strictly increase up any path (Cor. 19.5); a non-root's rank is frozen forever (Lemma 19.4); `rank ≤ ⌊lg n⌋` (Ex. 19.4-2); after compression `rank` is only an *upper bound* on height.
- **The proof shape:** potential method; `φ(x) = (α(n) − level(x))·x.rank − iter(x)` for non-roots, `α(n)·x.rank` for roots. `level` and `iter` measure how far `x` is from its parent in the `A`-hierarchy; both move monotonically, and any change drops `φ(x)` by `≥ 1`. A find path of `s` nodes has at most `α(n) + 2` nodes that *don't* drop, so `FIND-SET` is `O(α(n))` amortized.
- **The linked-list lesson worth keeping:** "always merge the smaller into the larger" ⟹ each element is touched `O(lg n)` times. That's **small-to-large merging**, and it's reusable far beyond this chapter.
- **Cannot do:** delete an element, split a set, undo a union (unless you drop compression), or tell you the *path* between two connected nodes.
- **Say it right:** *"union by rank plus path compression, `O(α(n))` amortized — effectively constant, at most 4 for any real input."*

Practice — where to drill this module

Union-find is the highest ratio of "trivial to write" to "solves hard problems" in the whole book. These are the drills.

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
`CONNECTED-COMPONENTS`, verbatim	547 · Number of Provinces	count components; the smallest possible DSU problem — `A1` as a submission
Grid components	200 · Number of Islands	DFS is the usual answer; do it with DSU too and compare
The **first** edge that closes a cycle	684 · Redundant Connection	`if FIND-SET(u) == FIND-SET(v)` is the entire solution — this is `A1` line 4
Components under a budget	1319 · Number of Operations to Make Network Connected	components minus one, and a redundant-edge count
Union-find over *names*	721 · Accounts Merge	the elements are strings, so you need an index map — the practical version of `MAKE-SET`
Constraints as equalities	990 · Satisfiability of Equality Equations	union all `==`, then check every `!=`; a two-pass DSU that reads like a proof
DSU inside Kruskal	1584 · Min Cost to Connect All Points · 1489 · Find Critical and Pseudo-Critical Edges in MST	the reason this module sits before M14
Weighted / relational DSU	399 · Evaluate Division	store a ratio to the parent; the `A4` variant, and a favourite follow-up

Beyond LeetCode. CSES Problem Set — *Graph Algorithms*. Codeforces `dsu` tag — the single best tag on the site for this module, and where rollback/offline DSU actually appear.

The drill that matters here: every time you reach for a DFS to answer "are these two things connected?", ask whether the connections *arrive over time*. If they do, DSU is `O(α)` per query and DFS is `O(V+E)` per query. Union-find is the **incremental** connectivity structure — that, and not the `α(n)` bound, is why it exists.

C++ Toolkit for This Module

Language material from Weiss, ***Data Structures and Algorithm Analysis in C++****, 4th ed., ch. 8 and §1.5.*

1. A forest in a `vector`, not in nodes

The pseudocode says `x.p` and `x.rank`, which sounds like a node with pointers. **Do not write that.** For `n` elements identified by integers `0..n-1`, two `vector<int>`s hold the whole structure:

```
vector<int> parent;   // parent[x] is x's parent; parent[x] == x
means x is a root
vector<int> rank_;    // an UPPER BOUND on the height of x's
subtree
```

No allocation per element, perfect cache locality, and the "pointer" is an index. Weiss builds the disjoint-set class exactly this way, and it is one of the clearest cases in the book where the array representation beats the pointer one outright.

2. `rank` is a reserved-ish name

`std::rank` is a type trait in `<type_traits>`, which `<bits/stdc++.h>` pulls in. With `using namespace std;` a member named `rank` is fine (members are found first), but a *local variable* or *free function* named `rank` can become ambiguous. The trailing underscore in `rank_` is the conventional fix — the same name-lookup hazard as M09's `mergeRuns`.

3. Recursive path compression and the stack

```
int findRecursive(vector<int>& parent, int x) {
    if (x != parent[x]) parent[x] = findRecursive(parent,
parent[x]);
    return parent[x];
}
```

This is CLRS's `FIND-SET` verbatim and it is beautiful — three lines, and the assignment on the way *back up* is what does the compression. But **the recursion depth is the tree height**, and before any compression has happened that can be `0(n)` with union-by-rank disabled. At `n ≈ 10⁶` that is a stack overflow. `A2` gives the two-pass iterative form and the one-line path-halving form; **use one of those in real code.**

4. `vector<int>` by reference, always

`int find(vector<int>& parent, int x)` — a **non-const** reference, because path compression *writes* to `parent`. That is the surprise in this structure: `FIND-SET` is a query that mutates. If you wrap it in a class, `find` cannot be a `const` member (or `parent` must be `mutable`, per M09 toolkit §4).

5. `iota` for the initial state

```
void initDemo(vector<int>& parent, int n) {
    parent.resize(n);
    iota(parent.begin(), parent.end(), 0);   // <numeric>: fills 0,
1, 2, ..., n-1
}
```

`iota(first, last, value)` fills a range with successively incremented values. It is exactly `MAKE-SET` applied to every element, in one line.

6. Structured bindings for the rollback stack

`A4`'s rollback DSU stores what each union changed so it can be undone. C++17 unpacks it cleanly:

```
struct Change { int child, parentBefore, rankRoot, rankBefore; };
void undoDemo(vector<Change>& history) {
    auto [c, pb, rr, rb] = history.back();   // four named values,
one line
    (void)c; (void)pb; (void)rr; (void)rb;
    history.pop_back();
}
```

Rollback DSU cannot use path compression, because compression rewrites parent pointers all over the tree and there is no cheap record of what it touched. That is why the rollback variant is `O(lg n)` per operation rather than `O(α)` — union by rank alone.

Appendix — C++ for Every Pseudocode Block

```
// The structure every entry below builds on. Two arrays, no nodes
(toolkit 1).
class DisjointSet {
public:
    explicit DisjointSet(int n) : parent_(n), rank_(n, 0),
count_(n) {
        iota(parent_.begin(), parent_.end(), 0);   // MAKE-SET for
every element
    }
    int find(int x);                    // A2
    bool unite(int x, int y);           // A3
    bool sameSet(int x, int y) { return find(x) == find(y); }   //
A1
    int components() const { return count_; }
private:
    vector<int> parent_;
    vector<int> rank_;      // trailing underscore: std::rank
exists (toolkit 2)
    int count_;             // number of disjoint sets, maintained
by unite()
};
```

A1 CONNECTED-COMPONENTS and SAME-COMPONENT

Pseudocode: §1, "Application: connected components".

```
// CONNECTED-COMPONENTS(G): one MAKE-SET per vertex, one UNION per
edge.
// Returns the structure, so the caller can answer SAME-COMPONENT
queries.
DisjointSet connectedComponents(int n, const vector<pair<int,int>>&
edges) {
    DisjointSet ds(n);                          // 1-2  for each
vertex v: MAKE-SET(v)
                                                //      (the
constructor's iota)
    for (const auto& [u, v] : edges) {          // 3  for each edge
(u,v)
        // 4  if FIND-SET(u) != FIND-SET(v)
        // 5      UNION(u, v)
        // unite() performs both the test and the union and reports
whether it
```

```
        // actually merged -- so the explicit comparison on line 4
disappears.
        // The RETURN VALUE is the useful part: `false` means u and
v were
        // ALREADY connected, i.e. this edge closes a CYCLE. That
single bit is
        // the whole solution to "Redundant Connection" and the
whole cycle test
        // inside Kruskal (M14).
        ds.unite(u, v);
    }
    return ds;                                  // return-by-value:
moved, not copied
}

// SAME-COMPONENT(u, v)
bool sameComponent(DisjointSet& ds, int u, int v) {
    return ds.find(u) == ds.find(v);          // 1-3
    // NOTE the non-const reference: find() COMPRESSES PATHS, so it
writes.
    // A query that mutates is unusual and is the reason this
cannot be
    // `const DisjointSet&` (toolkit 4).
}
```

Complexity. `Θ(V)` to initialise, then `E` unions. With union by rank **and** path compression the whole thing is `O((V + E) α(V))` — **effectively linear**.

Why not just DFS? For a *static* graph, DFS answers the same question in `Θ(V+E)` once. Union-find earns its place when the edges **arrive over time**: after each new edge you can answer connectivity queries in `O(α)`, where DFS would need a fresh `Θ(V+E)` traversal. **Union-find is the incremental connectivity structure.** Its one weakness is the mirror image: it supports no `DELETE`, because there is no cheap way to undo a merge (hence the rollback variant in `A4`).

A2 FIND-SET (with path compression)

Pseudocode: §*2, "Heuristic 2 — path compression"*.

```
// LITERAL translation of CLRS's two-pass recursive FIND-SET:
//
```

```
//   FIND-SET(x)
//   1  if x != x.p
//   2      x.p = FIND-SET(x.p)
//   3  return x.p
//
// The magic is on line 2: the assignment happens on the way BACK
UP, so every
// node on the path is re-pointed DIRECTLY at the root. The first
find is
// expensive; every later find from anywhere on that path is O(1).
//
// Do not ship this: the recursion depth is the tree height
(toolkit 3).
int findRecursive(vector<int>& parent, int x) {
    if (x != parent[x]) parent[x] = findRecursive(parent,
parent[x]);
    return parent[x];
}

// The same algorithm, iteratively: pass 1 walks to the root, pass
2 re-points
// everything on the path at it. Identical result, O(1) stack.
int DisjointSet::find(int x) {
    int root = x;
    while (root != parent_[root]) root = parent_[root];   // pass
1: find the root
    while (parent_[x] != root) {                          // pass
2: compress
        int next = parent_[x];
        parent_[x] = root;
        x = next;
    }
    return root;
}

// PATH HALVING -- one pass, and what most competitive code
actually uses.
// It points every OTHER node at its grandparent, halving the path
length as it
// goes. It gives the same O(alpha) amortized bound as full
compression, in one
// loop with no second pass and no extra memory.
```

```
int findHalving(vector<int>& parent, int x) {
    while (x != parent[x]) {
        parent[x] = parent[parent[x]];   // skip a generation
        x = parent[x];
    }
    return x;
}
```

Complexity. A single `FIND-SET` is `O(h)`, where `h` is the current tree height — **`Θ(lg n)` with union by rank alone, and `Θ(n)` with neither heuristic.** The amortized bounds are the point:

HEURISTICS	M OPERATIONS ON N ELEMENTS
neither	`Θ(m n)` worst case
union by rank only	`O(m lg n)`
path compression only	`Θ(n + f·(1 + log_{2+f/n} n))`
both	**`O(m α(n))`** (Theorem 19.14)

`α(n) ≤ 4` for every `n` you will ever meet — it is the inverse of the Ackermann function, and `α(n) ≤ 4` holds up to $A_4(1)$, a number vastly larger than the number of atoms in the observable universe. So the structure is **effectively constant time**, and "effectively" is doing almost no work in that sentence.

Path compression does not update `rank`. After compression a node's `rank` may badly overstate its height. That is fine and deliberate: `rank` is only ever used as an *upper bound* to decide which root to hang under, and the analysis (Lemma 19.4 onward) only needs it to be non-decreasing along a path to the root.

A3 MAKE-SET, UNION and LINK

Pseudocode: §2, "Full pseudocode".

```
// MAKE-SET(x): x.p = x; x.rank = 0.
// Done for every element at once in the constructor -- the iota
fills
// parent_ with 0,1,2,...  and rank_ is already all zeros.

// UNION(x, y) = LINK(FIND-SET(x), FIND-SET(y)), fused into one
function.
```

```
// Returns TRUE if a merge actually happened -- see the note in A1
on why that
// return value is the useful part.
bool DisjointSet::unite(int x, int y) {
    int rootX = find(x), rootY = find(y);          // UNION line 1:
FIND-SET both ends
    if (rootX == rootY) return false;              // already
together: nothing to do

    // ---- LINK(rx, ry), union BY RANK ----
    // Hang the SHORTER tree under the TALLER one, so the height
only grows when
    // the two are equal. This is what bounds the height at lg n
and is half of
    // the O(m alpha(n)) result.
    if (rank_[rootX] > rank_[rootY]) {             // 1  if x.rank
> y.rank
        parent_[rootY] = rootX;                    // 2      y.p =
x
    } else {
        parent_[rootX] = rootY;                    // 3  else x.p =
y
        if (rank_[rootX] == rank_[rootY])          // 4      if
x.rank == y.rank
            ++rank_[rootY];                      // 5
y.rank = y.rank + 1
        // Height increases ONLY on a tie -- and a tie at rank r
requires two
        // trees of >= 2^r nodes each, so rank r implies >= 2^r
nodes and rank
        // is at most lg n. That is Lemma 19.1, in one sentence.
    }
    --count_;                                  // one fewer disjoint
set
    return true;
}

// UNION BY SIZE -- the common alternative. Same O(alpha) bound,
and the size is
// often wanted anyway ("how big is this component?"), which rank
cannot tell you
// because path compression makes rank an over-estimate of height.
```

```
class DisjointSetBySize {
public:
    explicit DisjointSetBySize(int n) : parent_(n), size_(n, 1) {
        iota(parent_.begin(), parent_.end(), 0);
    }
    int find(int x) {                          // path halving (A2)
        while (x != parent_[x]) { parent_[x] = parent_[parent_[x]];
x = parent_[x]; }
        return x;
    }
    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return false;
        if (size_[rootX] < size_[rootY]) swap(rootX, rootY);   //
rx is now the LARGER root
        parent_[rootY] = rootX;
        size_[rootX] += size_[rootY];
        return true;
    }
    int setSize(int x) { return size_[find(x)]; }   // the reason
to prefer size
private:
    vector<int> parent_, size_;
};
```

Complexity. `MAKE-SET` `θ(1)`; `UNION` and `FIND-SET` `O(α(n))` amortized with both heuristics.

Union by rank alone gives Theorem 19.1: `O(m + n lg n)`. Neither heuristic alone is enough for `α`; the two together are, and that interaction is what makes the analysis (§4 of this module) so much harder than the code.

A4 Weighted and rollback DSU

These are not CLRS pseudocode — they are the two variants the practice problems above actually require, and both are one small change to `A3`.

```
// WEIGHTED (relational) DSU: alongside the parent pointer, store a
value
// relating each node to its parent -- a difference, a ratio, a
parity. This
```

```cpp
// answers "what is x relative to y?" and not merely "are they related?".
// LeetCode 399 (Evaluate Division) is exactly this, with ratios.
class WeightedDSU {
public:
    explicit WeightedDSU(int n) : parent_(n), rank_(n, 0), diff_(n, 0) {
        iota(parent_.begin(), parent_.end(), 0);
    }
    // Returns the root, and sets `weight` to (value[x] - value[root]).
    int find(int x, long long& weight) {
        if (x == parent_[x]) { weight = 0; return x; }
        long long parentOffset = 0;
        int root = find(parent_[x], parentOffset);
        // Path compression must ALSO compose the weights: x's offset to the
        // root is its offset to its parent PLUS the parent's offset to the root.
        // Forgetting this composition is the bug that makes weighted DSU
        // "mostly work" and then fail on deep trees.
        diff_[x] += parentOffset;
        parent_[x] = root;
        weight = diff_[x];
        return root;
    }
    // Assert value[y] - value[x] == d. Returns false if it contradicts what we
    // already know -- which is how these structures detect inconsistency.
    bool relate(int x, int y, long long wantedGap) {
        long long offsetX = 0, offsetY = 0;
        int rootX = find(x, offsetX), rootY = find(y, offsetY);
        if (rootX == rootY) return (offsetY - offsetX) == wantedGap;
        // Hang ry under rx and choose ry's offset so the equation holds.
        parent_[rootY] = rootX;
        diff_[rootY] = offsetX + wantedGap - offsetY;
        return true;
    }
```

```
private:
    vector<int> parent_, rank_;
    vector<long long> diff_;
};

// ROLLBACK DSU: supports undoing the last k unions, which plain
DSU cannot.
// Needed for offline dynamic connectivity and for "try an edge,
then take it
// back" searches.
//
// NO PATH COMPRESSION -- compression rewrites pointers all over
the tree with
// no cheap record of what it touched (toolkit 6). Union by rank
alone keeps the
// height at O(lg n), so every operation is O(lg n) rather than
O(alpha). That
// is the price of being able to undo.
class RollbackDSU {
public:
    explicit RollbackDSU(int n) : parent_(n), rank_(n, 0),
count_(n) {
        iota(parent_.begin(), parent_.end(), 0);
    }
    int find(int x) const {                      // const: it does
NOT mutate
        while (x != parent_[x]) x = parent_[x];
        return x;
    }
    bool unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) { history_.push_back({-1, -1, -1, -1});
return false; }
        if (rank_[rootX] > rank_[rootY]) swap(rootX, rootY);   //
rx is the shorter one
        history_.push_back({rootX, parent_[rootX], rootY,
rank_[rootY]});
        parent_[rootX] = rootY;
        if (rank_[rootX] == rank_[rootY]) ++rank_[rootY];
        --count_;
        return true;
    }
```

```
    void rollback() {
        if (history_.empty()) return;
        auto [child, parentBefore, rankRoot, rankBefore] =
history_.back();
        history_.pop_back();
        if (child == -1) return;                  // that call did
not merge
        parent_[child] = parentBefore;
        rank_[rankRoot] = rankBefore;
        ++count_;
    }
    int components() const { return count_; }
private:
    struct Change { int child, parentBefore, rankRoot, rankBefore;
};
    vector<int> parent_, rank_;
    vector<Change> history_;
    int count_;
};
```

Complexity. Weighted DSU: `O(α(n))`, same as plain — the weights ride along for free. Rollback DSU: **`O(lg n)` per operation**, and `O(1)` per rollback.

A5 Tarjan's offline LCA

Pseudocode: §5, "Tarjan's offline LCA".

```
// Answer many "lowest common ancestor of u and v" queries in ONE
DFS.
// "Offline" means all queries are known in advance -- which is
what lets a
// single traversal answer them all.
//
// THE IDEA: when the DFS finishes node u, every node already
coloured BLACK is
// in a DSU set whose `ancestor` is the LCA of that node and u. So
a query
// {u, v} is answered the moment the SECOND of the two is finished.
class OfflineLCA {
public:
    OfflineLCA(int n, const vector<vector<int>>& children)
```

```
        : children_(children), dsu_(n), ancestor_(n, 0), black_(n,
0) {}

    // queries[i] = {u, v}; returns answers[i] = LCA(u, v), or -1
if unrelated.
    vector<int> solve(int root, const vector<pair<int,int>>&
queries) {
        // Bucket each query under BOTH endpoints, so whichever
finishes second
        // finds it. This is the "for each node v such that {u,v}
in P" of line 8.
        queriesAt_.assign(children_.size(), {});
        for (int i = 0; i < (int)queries.size(); ++i) {

 queriesAt_[queries[i].first].push_back({queries[i].second, i});

 queriesAt_[queries[i].second].push_back({queries[i].first, i});
        }
        answers_.assign(queries.size(), -1);
        lca(root);
        return answers_;
    }
private:
    void lca(int u) {
        // MAKE-SET(u) is implicit: the constructor already did it
for every node.
        ancestor_[dsu_.find(u)] = u;              // 2  FIND-
SET(u).ancestor = u
        for (int v : children_[u]) {              // 3  for each
child v of u
            lca(v);                               // 4      LCA(v)
            dsu_.unite(u, v);                      // 5
UNION(u, v)
            // 6  FIND-SET(u).ancestor = u
            // MUST come after the union: the union may have
changed which node
            // is the root, and the ancestor field lives ON THE
ROOT.
            ancestor_[dsu_.find(u)] = u;
        }
        black_[u] = 1;                            // 7  u.color =
BLACK
```

```
        for (const auto& [v, queryIndex] : queriesAt_[u]) {   // 8
for each query {u, v}
            if (black_[v])                          // 9      if
v.color == BLACK
                answers_[queryIndex] = ancestor_[dsu_.find(v)];
// 10
        }
    }
    vector<vector<int>> children_;
    DisjointSet dsu_;
    vector<int> ancestor_;
    vector<char> black_;
    vector<vector<pair<int,int>>> queriesAt_;
    vector<int> answers_;
};
```

Complexity. `O((V + Q) α(V))` **— effectively linear in the tree size plus the number of queries.** Compare the online alternatives: binary lifting is `O(V lg V)` preprocessing and `O(lg V)` per query; Euler tour + sparse table is `O(V lg V)` and `O(1)`. **Tarjan wins when you can see all the queries up front, and is useless when you cannot.**

Note the recursion: `lca` recurses to the tree's depth, so on a path-shaped tree of 10^6 nodes it overflows the stack (toolkit §3). An explicit stack is required for large inputs.

A6 OFFLINE-MINIMUM

Pseudocode: §*5, "Offline minimum".*

```
// Given a sequence of INSERT and EXTRACT-MIN operations on {1..n}
known in
// advance, report which key each EXTRACT-MIN returns -- WITHOUT
ever building
// a priority queue.
//
// `sequence` is the operation list: a positive value means INSERT
that key,
// 0 means EXTRACT-MIN.
//
// THE IDEA: process keys in INCREASING order. Key i must be
returned by the
```

```cpp
// FIRST not-yet-used extraction that comes after i was inserted.
Union-find
// tracks "the first still-available extraction at or after
position j" by
// merging each used slot into the next one -- a classic "next free
slot"
// pattern that appears far beyond this problem.
vector<int> offlineMinimum(const vector<int>& sequence, int n) {
    // Split into m blocks K_1..K_{m+1}: K_j is the set of keys
inserted before
    // the j-th extraction. Block m+1 collects keys never
extracted.
    vector<int> blockOf(n + 1, 0);
    int extractCount = 0;
    {
        int openBlock = 1;
        for (int op : sequence) {
            if (op == 0) ++openBlock, ++extractCount;        // an
EXTRACT-MIN closes this block
            else blockOf[op] = openBlock;         // this key
belongs to the open block
        }
    }
    const int last = extractCount + 1;

    // DSU over blocks 1..m+2. find(j) = the smallest l >= j whose
block still
    // exists, i.e. the first extraction not yet assigned an
answer.
    vector<int> nextSurviving(last + 2);
    iota(nextSurviving.begin(), nextSurviving.end(), 0);
    function<int(int)> find = [&](int block) {
        while (block != nextSurviving[block]) {
nextSurviving[block] = nextSurviving[nextSurviving[block]]; block =
nextSurviving[block]; }   // path halving
        return block;
    };

    vector<int> extracted(last + 1, 0);      // extracted[j] = key
returned by
                                             // the j-th EXTRACT-
MIN, 0 if none
```

```
    for (int i = 1; i <= n; ++i) {           // 1  for i = 1 to n
(INCREASING keys)
        if (blockOf[i] == 0) continue;       //    key never
inserted
        int block = find(blockOf[i]);            // 2  determine j
such that i in K_j
        if (block != last) {                     // 3  if j != m+1
            extracted[block] = i;                // 4
extracted[j] = i
            // 5-6  merge K_j into the next surviving block: this
extraction is
            //      now used up, so future lookups skip past it.
            nextSurviving[block] = find(block + 1);
        }
    }
    extracted.resize(extractCount + 1);
    return extracted;                        // 7  return extracted
(1-indexed)
}
```

Complexity. `O((m + n) α(n))` — effectively linear, against `O(n lg n)` for actually running a heap.

Why this is worth the page. It is the cleanest example of union-find used for something that is **not connectivity at all**. The sets here are *blocks of an operation sequence*, and `UNION` means "this slot is used up; skip to the next one". Once you have seen this "point to the next free slot" pattern you start finding it everywhere — interval assignment, scheduling to the first free machine, and the offline version of half the problems that otherwise need a `set<int>` and `lower_bound`.

Next: M11 — Dynamic Programming (CLRS 14 + Skiena 10) — optimal substructure, overlapping subproblems, memoization vs. bottom-up, and the recognition skill that decides most interviews.